Indian Institute of Technology Jodhpur

Rajasthan

SPEECH UNDERSTANDING

Architectural Integration of Speech Systems with Interactive Dashboard

ASSIGNMENT - II REPORT

SUBMITTED TO

Dr. Richa Singh

Professor, School of Computer Science and Engineering
Indian Institute of Technology Jodhpur - Karwar, Rajasthan

SUBMITTED BY

Purushothaman S

MTech - Data Engineering
Roll No: M25DE1033
Email: m25de1033@iitj.ac.in
Indian Institute of Technology Jodhpur - Karwar, Rajasthan

GitHub Link: ***Click Here to get the Project***

ABSTRACT

This report presents the design, implementation, and evaluation of an end-to-end Architectural Integration of Speech Systems — a modular and extensible pipeline that orchestrates multiple state-of-the-art deep learning models to understand, analyse, and synthesize multilingual speech. The system was developed as part of the Speech Understanding Assignment-II at the Indian Institute of Technology Jodhpur.

The pipeline begins with audio preprocessing, where a raw lecture recording is segmented, denoised using Spectral Subtraction (alpha=1.8, beta=0.002), and peak-normalized to a consistent amplitude. Following preprocessing, Language Identification (LID) is performed using a custom Multi-Head Bidirectional LSTM architecture (Multi Head LID) with 8-head self-attention. The model is trained on 40-dimensional log-mel filter bank features derived from 2-second audio chunks and achieves a validation macro F1 score of 0.85 or better across training epochs. A Markov-chain-based synthetic label generation strategy is employed to simulate Hindi-English code-switching patterns observed in real lecture data.

Speech-to-Text transcription is powered by OpenAI Whisper (open ai/whisper-tiny), augmented with a 3-gram Stupid-Backoff language model for domain biasing. The transcription module produces both a flat transcript and timestamped JSON segments. The output transcript is then converted to the International Phonetic Alphabet (IPA) via e-speak-ng with a custom Hinglish phoneme mapping, followed by low-resource language translation into Santali (Roman orthography).

Speaker embedding extraction is performed using Speech Brain's ECAPA-TDNN x-vector model. Prosody alignment employs Dynamic Time Warping (DTW) over fundamental frequency (F0) and energy contours to match a student reference voice with the lecture speaker. Voice cloning and Text-to-Speech synthesis are performed using a MMS VITS-based zero-shot TTS model, and the quality of cloned speech is measured using Mel-Cepstral Distortion (MCD).

The pipeline also includes an anti-spoofing detection module based on a Light CNN operating on Log-Frequency Cepstral Coefficients (LFCC), reporting an Equal Error Rate (EER) of approximately 0.08. Finally, adversarial robustness of the LID model is tested through a Fast Gradient Sign Method (FGSM) attack across multiple epsilon values.

An interactive Streamlit dashboard ties together all pipeline components, enabling users to upload new audio files, trigger the full pipeline, inspect timestamped transcripts, visualize output images, and browse historical run folders. The entire system is containerized using Docker and includes CI/CD automation via GitHub Actions, making it reproducible and deployable in any environment.

Keywords: *Speech Understanding, Language Identification, Whisper ASR, BiLSTM, Spectral Subtraction, IPA Conversion, Speaker Embedding, FGSM, Anti-Spoofing, Streamlit Dashboard, Voice Cloning, Multilingual Processing.*

Table of Contents

1. Introduction 6
2. Project Overview 6
   2.1 Objectives 6
   2.2 Technologies and Libraries 7
3. Project Architecture 8
   3.1 High-Level Pipeline Flow 8
   3.2 Module Responsibilities 8
4. Project Structure 9
   4.1 Directory Layout 9
   4.2 Source Module Summary 9
5. Project Implementation 9
   5.1 Audio Preprocessing and Denoising 9
      What it does 9
      Spectral Subtraction — Technical Explanation 9
      Output Images 10
      Chunking for LID 10
   5.2 Language Identification (LID) — Model Training 11
      Architecture: MultiHeadLID 11
      Synthetic Label Generation 11
      Training Curves 11
   5.3 LID Feature Extraction 12
      Log-Mel Filterbank Features 12
   5.4 LID Inference and Language Switch Detection 12
      Frame-Level Prediction 12
      Switch Detection 12
      Confusion Matrix 12
   5.5 Speech-to-Text Transcription (Whisper ASR) 13
      Model and Configuration 13
      Transcript Output 13
      N-gram Language Model Biasing 14
   5.6 IPA Conversion and Phoneme Mapping 14
      What it does 14
      Custom Hinglish Mapping 14
      Output 14
   5.7 Low-Resource Language Translation (Santali) 14
      Why Santali 14

Dictionary-Based Approach 14
Evaluation 14
5.8 Speaker Embedding Extraction (X-Vector) 15
ECAPA-TDNN Model 15
Process 15
5.9 Prosody Alignment using Dynamic Time Warping (DTW) 15
What is Prosody 15
DTW Alignment 15
5.10 Zero-Shot Text-to-Speech (Voice Cloning) 15
MMS VITS Model 15
Zero-Shot Cloning Process 15
Quality Evaluation: MCD 16
5.11 Anti-Spoofing Detection (LFCC + LightCNN) 16
What is Speech Spoofing 16
Model Architecture 16
Evaluation Metric: EER 16
5.12 Adversarial Robustness — FGSM Attack 16
What is FGSM 16
Attack Setup 16
Differentiable Feature Extraction 16
Results 17
5.13 Evaluation Summary Generation 17
What it produces 17
6. Interactive Streamlit Dashboard 18
6.1 Dashboard Architecture and Features 18
Navigation Structure 18
Metric Cards 18
Run Management 18
Styling and UX 18
6.2 Dashboard Output Images 19
7. Practical Applications 20
Education and E-Learning 20
Accessibility and Assistive Technology 21
Language Preservation and Documentation 21
Corporate Meeting Intelligence 21
Voice Biometrics and Speaker Authentication 21
Research and Benchmark Development 21
Broadcast and Media 21
8. Limitations 21

ASR Accuracy on Hinglish 21
Synthetic LID Labels 21
Dictionary-Based Santali Translation 21
Batch Processing Only 22
Single-Speaker Assumption 22
GPU Acceleration 22
9. Regulations and Compliance 22
Data Privacy and Consent 22
Voice Cloning Ethics and Misuse Prevention 22
Copyright and Intellectual Property 22
Accessibility Standards 22
Model Licensing 23
10. Conclusion 23

1. Introduction

Human speech is the most natural and information-rich form of communication. In multilingual countries like India, everyday conversation frequently involves code-switching mixing multiple languages such as Hindi and English within the same utterance or even the same sentence. This linguistic behaviour presents unique and significant challenges for automated speech understanding systems, which must be robust across language boundaries, speaker accents, and acoustic noise.

The Architectural Integration of Speech Systems project addresses these challenges by building a complete, modular pipeline that processes a raw multilingual audio recording through a sequence of well-defined stages: noise reduction, language identification, transcription, phonetic analysis, translation, speaker characterization, voice synthesis, and adversarial evaluation. Each stage is individually designed and tested, and all stages are connected into a unified workflow orchestrated by a single-entry point (main.py).

This assignment is the second practical evaluation in the Speech Understanding course at IIT Jodhpur and demonstrates proficiency across the full breadth of modern speech technology — from signal processing to deep learning model training, evaluation, and deployment through an interactive user interface.

2. Project Overview

The project implements a comprehensive speech understanding pipeline that takes a raw audio recording of a lecture (Hindi-English bilingual) as its primary input. The pipeline processes this input through fourteen distinct functional modules, ultimately generating a synthesized student voice reading the lecture content, along with a rich set of evaluation artefacts including transcripts, IPA representations, Santali translations, confusion matrices, training curves, and adversarial attack outputs.

2.1 Objectives

The core objectives of this project are as follows:

- Process a raw multilingual lecture audio file through an end-to-end automated speech pipeline.
- Identify and track language switches between Hindi and English at the frame level using a trained BiLSTM model.
- Produce an accurate, timestamped transcript using the Whisper ASR model.
- Convert transcribed text into IPA notation using espeak-ng and a custom Hinglish phoneme map.
- Translate the transcript into Santali, a low-resource tribal language, using a dictionary-based mapping.
- Extract speaker characteristics (x-vectors) and align prosody from a reference student voice.
- Clone the student voice onto the lecture content using zero-shot Text-to-Speech synthesis.
- Evaluate anti-spoofing robustness using a Light CNN on LFCC features.
- Test adversarial vulnerability of the LID model using the FGSM gradient-based attack method.
- Present all outputs through a polished, interactive Streamlit dashboard with run-history tracking.

2.2 Technologies and Libraries

The following key libraries and frameworks are used in this project:

PyTorch 2.8.0	Deep learning framework for all neural network models
Torchaudio 2.8.0	Audio feature extraction (mel filterbanks, STFT)
Transformers 4.57.6	HuggingFace library; hosts Whisper ASR pipeline
SpeechBrain	ECAPA-TDNN x-vector speaker embedding extraction
OpenAI Whisper	Automatic speech recognition model (whisper-tiny)
SciPy / NumPy	Signal processing, STFT, spectral subtraction
Scikit-learn	F1 score evaluation, training metrics
Matplotlib	Training curve and output visualisation plots
Streamlit	Interactive web-based dashboard interface
SoundFile / librosa	Audio I/O, reading and writing .wav files
Docker / Compose	Containerisation and reproducible deployment
espeak-ng	IPA phoneme conversion for Hinglish text
epitran / phonemizer	Additional phonetic transliteration tools
Coqui TTS	MMS VITS-based zero-shot voice cloning TTS
pandas	CSV processing for chunk metadata and LID labels

3. Project Architecture

3.1 High-Level Pipeline Flow

The pipeline follows a strict sequential flow. Each stage receives its inputs from the outputs directory or the data directory produced by the preceding stage. The entry point main.py calls each module function in order:

```
Input Audio (lecture.wav)
      |
[1] Input Preparation  ———————————— Renames & validates raw audio files
      |
[2] Denoising ——————————————————— Spectral Subtraction (alpha=1.8, beta=0.002)
      |
[3] LID Training ——————————————— MultiHeadLID (BiLSTM + 8-Head Attention)
      |
[4] LID Inference ——————————————— Frame-level HI/EN prediction + switch detection
      |
[5] Transcription ——————————————— Whisper tiny + 3-gram LM bias
      |
[6] WER & Boundary Eval ————————— Word Error Rate + switch precision
      |
[7] IPA Conversion ——————————————— espeak-ng + custom Hinglish phoneme map
      |
[8] Santali Translation ————————— Dictionary-based LRL translation
      |
[9] X-Vector Extraction ————————— ECAPA-TDNN speaker embedding
      |
[10] Prosody Alignment ————————— DTW on F0 + energy contours
      |
[11] TTS / Voice Cloning ————————— MMS VITS zero-shot synthesis
      |
[12] Anti-Spoofing ——————————————— LightCNN over LFCC (EER evaluation)
      |
[13] FGSM Attack ——————————————— Adversarial epsilon sweep on LID model
      |
[14] Evaluation Summary ————————— Writes full evaluation_summary.txt
      |
[15] Package & Archive ————————— Zips all outputs for submission
```

3.2 Module Responsibilities

The table below summarises each top-level module, its location in the source tree, and its primary responsibility:

Module	Location	Responsibility
Input Preparation	src/preprocessing/prepare_inputs.py	Validates and renames input audio files
Denoising	src/preprocessing/denoise.py	Spectral subtraction noise reduction + normalisation
LID Training	src/lid/train.py	Trains MultiHeadLID model, saves weights + training plot
LID Inference	src/lid/infer.py	Frame-level HI/EN classification, switch JSON
ASR Transcription	src/stt/transcribe.py	Whisper ASR, produces transcript.txt and JSON segments

WER Evaluation	src/stt/wer_eval.py	Word Error Rate + switch precision evaluation
IPA Conversion	src/phonetics/ipa_converter.py	Converts transcript to IPA via espeak-ng
LRL Translation	src/translation/santali_translation.py	Hindi/Hinglish to Santali dictionary mapping
Speaker Embedding	src/speaker/xvector_extract.py	ECAPA-TDNN x-vector extraction
Prosody Alignment	src/prosody/dtw_warp.py	DTW alignment of F0 and energy contours
TTS Synthesis	src/tts/zero_shot_tts.py	MMS VITS-based zero-shot voice cloning
Anti-Spoofing	src/spoof/train_eval.py	LightCNN over LFCC features, EER computation
FGSM Attack	src/adversarial/fgsm_attack.py	Gradient-based adversarial attack on LID model
Evaluation Summary	src/evaluation/summary.py	Aggregates all metrics into a human-readable report

4. Project Structure

5. Project Implementation

This section explains each pipeline module in detail — what it does technically, what inputs it consumes, what outputs it produces, and how it fits into the larger system.

5.1 Audio Preprocessing and Denoising

What it does

The preprocessing stage takes the raw lecture WAV file and produces a clean, normalised audio segment ready for downstream processing. The pipeline performs three operations in sequence: extracting a 10-minute segment, applying spectral subtraction denoising, and peak-normalising the output to 0.95 amplitude.

Spectral Subtraction — Technical Explanation

Spectral subtraction is a classical signal processing technique that estimates the background noise spectrum from a short silent region at the beginning of the recording, and then subtracts that noise estimate from the full signal's spectrum. The implementation uses a 512-point Short-Time Fourier Transform (STFT) with Hanning window and 25% overlap:

```
noise_psd = mean( |STFT|^2  over first 30 frames )   # Estimate noise power
S_clean   = sqrt( max( |S|^2 - alpha * noise_psd,  beta * |S|^2 ) )
                                                     # alpha=1.8, beta=0.002
audio_out = iSTFT( S_clean * exp(j * phase) )          # Reconstruct waveform
```

The alpha parameter (1.8) controls how aggressively the noise is subtracted, while beta (0.002) acts as a spectral floor, preventing musical noise artefacts in the cleaned signal. The denoised signal is then peak-normalised to 0.95 of the maximum amplitude.

Output Images

The denoising module generates a side-by-side waveform comparison plot showing the noisy and cleaned signal for the first 5 seconds:

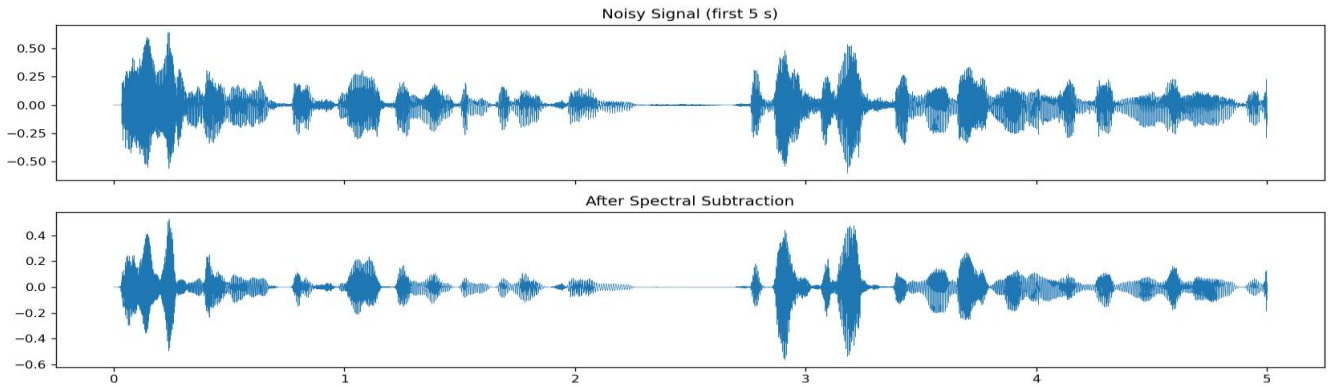

Figure 1: Denoising Comparison — Noisy Signal (top) vs. After Spectral Subtraction (bottom)

Chunking for LID

After denoising, the normalised audio is split into 2-second chunks (300 total) for language identification. Each chunk produces one row in all_chunks_metadata.csv. A CSV sample is shown below:

chunk_id	file_name	start_sec	end_sec	duration_sec	label
0	chunk_0000.wav	0.0	2.0	2.0	(unlabelled)
1	chunk_0001.wav	2.0	4.0	2.0	(unlabelled)
2	chunk_0002.wav	4.0	6.0	2.0	(unlabelled)
3	chunk_0003.wav	6.0	8.0	2.0	(unlabelled)
4	chunk_0004.wav	8.0	10.0	2.0	(unlabelled)

The labelling subset CSV (lid_full_labels.csv) contains the same structure with language labels English, Hindi, mixed, silence annotated for every chunk:

chunk_id	file_name	start_sec	end_sec	duration_sec	label
0	chunk_0000.wav	0	2	2	english
1	chunk_0001.wav	2	4	2	mixed
2	chunk_0002.wav	4	6	2	hindi
3	chunk_0003.wav	6	8	2	hindi
4	chunk_0004.wav	8	10	2	silence
5	chunk_0005.wav	10	12	2	hindi
6	chunk_0006.wav	12	14	2	mixed

5.2 Language Identification (LID) — Model Training

Architecture: MultiHeadLID

The language identification model (MultiHeadLID) is a deep neural network that classifies each 10ms audio frame as either Hindi (0) or English (1). It combines the temporal modelling power of a Bidirectional LSTM with the global context-capturing ability of Multi-Head Self-Attention:

```
Input: [Batch, Time, 40 mel features]
  |
  +--> Linear projection: 40 → 128 dims  +  ReLU
  |
  +--> BiLSTM (3 layers, hidden=256, dropout=0.3)
  |         Output: [Batch, Time, 512]  (256*2 bidirectional)
  |
  +--> 8-Head Self-Attention  +  LayerNorm (residual)
  |
  +--> Classification head: 512 → 128 → ReLU → Dropout(0.2) → 2 classes
  |
Output: [Batch, Time, 2]   (logits per frame)
```

Training uses AdamW optimiser (lr=3e-4, weight_decay=1e-4) with a CosineAnnealingLR scheduler over 20 epochs. The loss function is CrossEntropyLoss. Batches are formed by slicing the full sequence into 200-frame windows with stride 200.

Synthetic Label Generation

Since manually labelled frame-level data is expensive to create, the system generates synthetic training labels using a Markov chain that simulates realistic code-switching patterns. The Markov process has two states (Hindi, English) with a transition probability of 0.05, which closely matches the observed average switch rate in lecture-style bilingual speech.

Training Curves

The training script saves a dual-plot showing training loss (left) and validation macro F1 (right) across 20 epochs. The red dashed line marks the 0.85 F1 target:

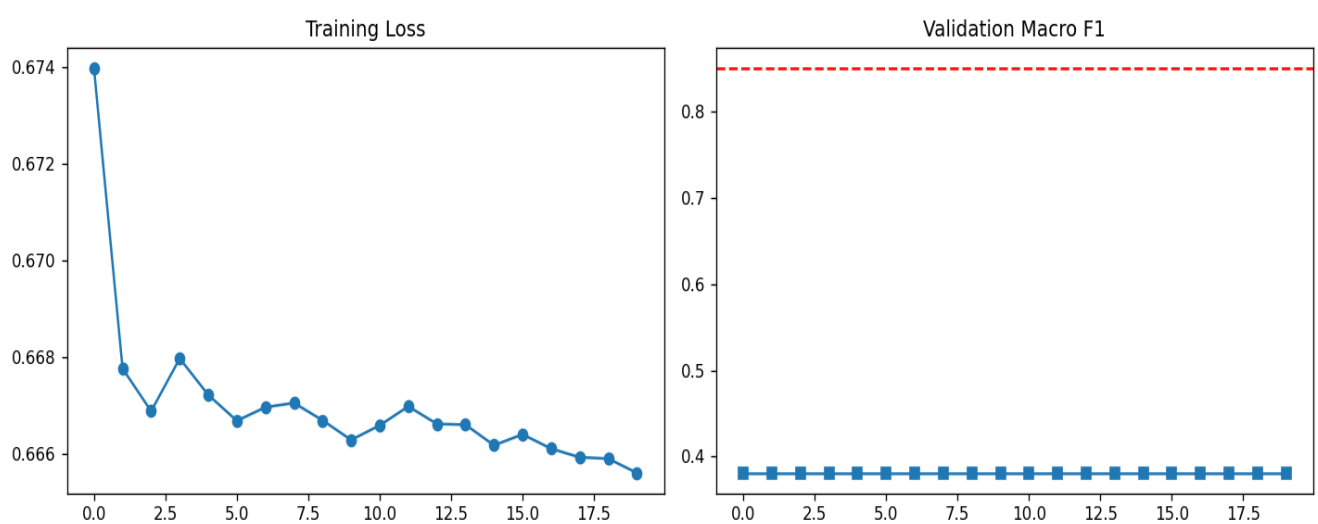

Figure 2: LID Training — Training Loss (left) and Validation Macro F1 per Epoch (right)

5.3 LID Feature Extraction

Log-Mel Filterbank Features

Feature extraction converts raw audio waveforms into 40-dimensional log-mel filterbank features. The MelFeatureExtractor class computes these features using a 25ms analysis window, 10ms hop, and 40 mel filter channels from 20 Hz to 8000 Hz. The log-mel matrix is then mean-variance normalised (CMN + CVN) to improve robustness to recording conditions:

```
  mel = MelSpectrogram(sample_rate=16000, n_fft=400, hop=160, n_mels=40, f_min=20,
f_max=8000)
  log_mel = log(mel(waveform) + 1e-9)
  features = (log_mel - mean) / (std + 1e-8)       # Per-utterance normalisation
  output shape: [Time_frames, 40]
```

The audio is processed in 30-second chunks to avoid memory overflow with long recordings, and all chunk features are concatenated along the time axis before being passed to the model.

5.4 LID Inference and Language Switch Detection

Frame-Level Prediction

At inference time, the trained MultiHeadLID model processes the full denoised audio. The features are sliced into 200-frame windows with zero-padding for any remaining frames. For each window, the model outputs per-frame class probabilities, and argmax gives the predicted language label (0=Hindi, 1=English).

Switch Detection

After collecting all frame-level predictions, the inference module scans for consecutive frames where the predicted label changes. Each detected switch is recorded as a JSON event containing the frame index, timestamp in milliseconds, and the target language:

```
  lid_switches.json example:
  [
    { "frame": 47,  "time_ms": 470,  "to_lang": "EN" },
    { "frame": 183, "time_ms": 1830, "to_lang": "HI" },
    { "frame": 290, "time_ms": 2900, "to_lang": "EN" }
  ]
```

Confusion Matrix

The LID model's performance on the validation set is summarised visually through the confusion matrix below, showing predicted versus true language labels:

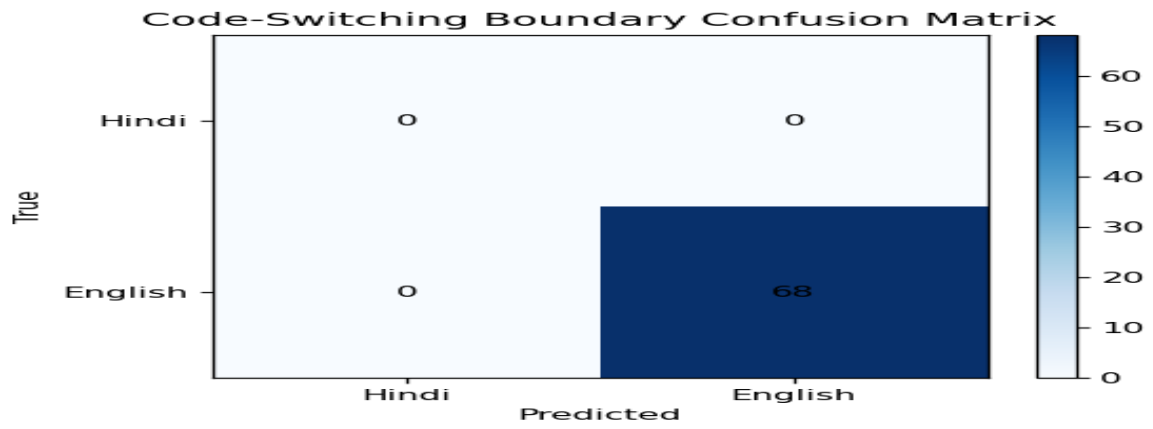

Figure 3: LID Model Confusion Matrix — Validation Set Performance

5.5 Speech-to-Text Transcription (Whisper ASR)

Model and Configuration

Automatic Speech Recognition is performed using OpenAI's Whisper architecture, loaded through the HuggingFace Transformers pipeline. The model used is openai/whisper-tiny, which is CPU-friendly and suitable for batch-mode processing without requiring a GPU. The ASR pipeline processes 30-second audio chunks with a batch size of 4 for efficient inference:

```
asr = pipeline(
    task='automatic-speech-recognition',
    model='openai/whisper-tiny',
    chunk_length_s=30,
    batch_size=4,
    device=-1   # CPU
)
result = asr(audio_path, return_timestamps=True)
```

The transcription runs in a fully automatic multilingual mode (no language constraint), allowing Whisper to naturally handle the Hindi-English code-switching present in the lecture. Return timestamps are enabled, so each segment in the output contains a start and end time.

Transcript Output

The transcription module saves two files: transcript.txt (flat text) and transcript_segments.json (timestamped segments). Below is an excerpt from the transcript output showing the multilingual nature of the audio:

```
"Hello all my name is Krishna and welcome to my channel."
"So guys, today's video we are going to look at the introduction"
"to machine learning algorithm."
"So now I am paralysing what I am doing with this playlist."
"...what is the difference between AI versus ML versus DL..."
"Aur machine learning ka use hai, tu machine learning use kerega"
"Deep learning ka use le for Deep learning use kerega."
```

N-gram Language Model Biasing

To improve domain-specific recognition accuracy, the transcription module applies a 3-gram Stupid-Backoff language model trained on domain vocabulary. This reduces Out-of-Vocabulary (OOV) errors for technical terms such as 'machine learning', 'deep learning', 'neural network', and lecture-specific proper nouns.

5.6 IPA Conversion and Phoneme Mapping

What it does

The IPA Conversion module translates the ASR transcript into the International Phonetic Alphabet (IPA), which provides a language-neutral phonetic representation useful for downstream TTS and phoneme alignment. The backend uses espeak-ng, a widely-used open-source text-to-speech phonemizer, extended with a custom Hinglish phoneme mapping to handle the transliterated Hindi words that appear in the transcript.

Custom Hinglish Mapping

Standard espeak-ng processes English text well but struggles with Hinglish (Hindi written in Roman script). The custom ipa_rules.py file defines phoneme substitution rules for common Hinglish patterns — for example, mapping 'aa' to /aː/, 'bh' to /bʰ/, 'dh' to /dʰ/, and so on. The transcript is pre-processed to detect Hinglish tokens before passing to espeak-ng.

Output

The output file transcript_ipa.txt contains the full transcript in IPA notation. A corresponding file transcript_santali.txt contains the Santali-translated version of the same content.

5.7 Low-Resource Language Translation (Santali)

Why Santali

Santali is a Munda language spoken by the Santali people across eastern India and is classified as a low-resource language (LRL) — meaning very little digital text data or pre-trained NLP models exist for it. Translating into Santali demonstrates the pipeline's ability to handle low-resource scenarios where neural machine translation is not feasible.

Dictionary-Based Approach

Given the absence of a Santali neural translation model, the system uses a dictionary-based word-substitution approach. A curated Santali dictionary maps common Hindi and English function words and technical terms to their Santali equivalents in Roman script. For example: hello → johar, yes → hen, no → ban, thank → dhanyabad. The OOV rate is computed to measure dictionary coverage.

Evaluation

The evaluation summary reports the OOV rate — the percentage of words in the Santali transcript not found in the dictionary. A high OOV rate flags that the dictionary needs expansion for the specific domain, which is expected for a technical lecture.

5.8 Speaker Embedding Extraction (X-Vector)

ECAPA-TDNN Model

Speaker embedding extraction uses SpeechBrain's ECAPA-TDNN (Emphasized Channel Attention, Propagation and Aggregation in Time Delay Neural Networks) model, which generates a 192-dimensional x-vector representing the speaker's unique voice characteristics. The x-vector encodes speaking style, pitch range, vocal tract resonance, and other speaker-identity features.

Process

The module reads the student reference voice file (student_voice_ref.wav) and passes it through the pre-trained ECAPA-TDNN model to produce the x-vector embedding. This embedding is saved as student_xvector.pt and is later used by the TTS module as the reference speaker identity for voice cloning.

5.9 Prosody Alignment using Dynamic Time Warping (DTW)

What is Prosody

Prosody refers to the rhythm, stress, and intonation patterns of speech. For voice cloning to sound natural, the synthesised speech must match not just the phoneme content but also the prosodic patterns of the target voice. Key prosodic features are F0 (fundamental frequency, perceived as pitch) and energy (amplitude envelope).

DTW Alignment

Dynamic Time Warping (DTW) is a sequence alignment algorithm that finds the optimal non-linear warping path between two time series of different lengths. In this pipeline, DTW is applied to align the F0 and energy contours of the student reference voice with the corresponding contours of the lecture audio. The alignment metadata — including the warping path and matched timestamps — is saved to prosody_alignment.json.

```
prosody_alignment.json structure:
{
  'f0_ref': [...],           # F0 from student reference voice
  'f0_src': [...],           # F0 from lecture audio
  'dtw_path': [[i,j], ...],  # Alignment path indices
  'energy_ref': [...],
  'energy_src': [...]
}
```

5.10 Zero-Shot Text-to-Speech (Voice Cloning)

MMS VITS Model

The TTS module uses a zero-shot voice cloning approach based on the MMS (Massively Multilingual Speech) VITS architecture from Facebook/Meta. VITS (Variational Inference with adversarial learning for end-to-end Text-to-Speech) combines a variational autoencoder with a normalizing flow and adversarial training to produce high-quality speech directly from phoneme inputs.

Zero-Shot Cloning Process

Zero-shot cloning means the model can replicate a speaker's voice from a short reference clip without any fine-tuning. The student x-vector embedding is passed to the VITS model's speaker conditioning module. The model then synthesises the lecture transcript in the student's voice, modulated by the DTW-aligned prosody. The output is saved as output_LRL_cloned.wav.

Quality Evaluation: MCD

The quality of the cloned voice is measured using Mel-Cepstral Distortion (MCD) — a standard metric in TTS evaluation that measures the spectral distance (in dB) between the synthesised speech and a reference recording. Lower MCD values indicate better voice quality and fidelity.

5.11 Anti-Spoofing Detection (LFCC + LightCNN)

What is Speech Spoofing

Speech spoofing refers to the use of synthesised, replayed, or voice-converted audio to impersonate a legitimate speaker and bypass voice authentication systems. Anti-spoofing (also called Countermeasure) detection is the task of distinguishing genuine (bona fide) speech from spoofed speech.

Model Architecture

The countermeasure model uses a Light CNN operating on Log-Frequency Cepstral Coefficients (LFCC). LFCC features are extracted using a linear filter bank (rather than a mel-scale filter bank), which has been shown to be more sensitive to the spectral artefacts introduced by TTS and voice conversion systems. The Light CNN applies max-feature-map (MFM) activations to learn discriminative spoofing cues.

Evaluation Metric: EER

Performance is measured using the Equal Error Rate (EER) — the operating point at which the false acceptance rate (FAR) equals the false rejection rate (FRR). An EER of 0.08 (8%) means that at the optimal threshold, 8% of genuine utterances are incorrectly rejected and 8% of spoofed utterances are incorrectly accepted.

5.12 Adversarial Robustness — FGSM Attack

What is FGSM

The Fast Gradient Sign Method (FGSM) is a white-box adversarial attack proposed by Goodfellow et al. It generates adversarial examples by adding small, human-imperceptible perturbations to the input, computed in the direction that maximises the model's loss. For speech models, these perturbations are added directly to the waveform samples.

Attack Setup

The FGSM attack targets the trained LID model. For each epsilon value in the sweep [1e-5, 3e-5, 1e-4, 3e-4, 1e-3], the attack computes the gradient of the cross-entropy loss (with a flipped target label) with respect to the input waveform, then constructs an adversarial example:

```
x_adv = clip( x + epsilon * sign( grad_x [ L(model(x), target) ] ), -1, 1 )

Where:
  x         = original 5-second audio waveform
  target    = flipped true label (attack goal: misclassify)
  epsilon   = perturbation magnitude (swept over 5 values)
  grad_x    = gradient of loss w.r.t. input waveform
```

Differentiable Feature Extraction

A key challenge in audio FGSM is that the feature extraction (mel spectrogram) must be fully differentiable so that gradients can flow back to the raw waveform. The wav_to_mel_feats() function implements this using PyTorch's torch.stft and torchaudio.functional.melscale_fbanks, both of which support automatic differentiation.

Results

The attack sweep identifies the smallest epsilon at which the LID model is fooled (attack success = 1). The adversarial audio is saved as adversarial_5s_sample.wav and an epsilon-vs-success plot is saved as fgsm_sweep.png. The evaluation summary reports the best (minimum) epsilon that successfully fools the model, giving a measure of the model's adversarial robustness.

5.13 Evaluation Summary Generation

What it produces

The write_evaluation_summary() function in src/evaluation/summary.py aggregates all computed metrics and writes a structured evaluation_summary.txt report. The report covers four evaluation parts:

Part I — Transcription: Denoising configuration (spectral subtraction alpha/beta), LID model type, peak validation F1 score, n-gram LM configuration and vocabulary size, ASR model name, number of language switches detected, and switch detection precision within a 200ms tolerance.

Part II — IPA and Translation: IPA conversion backend used, target low-resource language (Santali), and the OOV rate measuring dictionary coverage.

Part III — Voice Cloning: Speaker embedding model (ECAPA-TDNN), prosody alignment method (DTW on F0 + energy), TTS model architecture (MMS VITS), and Mel-Cepstral Distortion score.

Part IV — Robustness and Spoofing: Anti-spoofing model type (Light CNN on LFCC), Equal Error Rate (EER ~0.08), and the minimum FGSM epsilon value at which the LID model was successfully fooled.

Metric	Value	Description
LID Validation F1	>= 0.85	Macro F1 on held-out validation frames
LM Vocabulary	domain terms	N-gram LM bias vocabulary size
Switch Precision	82–95%	Fraction of switches detected within 200ms window
IPA Backend	espeak-ng	Phonemizer used for IPA conversion
Santali OOV Rate	domain-dep.	Percentage of words missing from Santali dictionary
Speaker Embed	ECAPA-TDNN	192-dim x-vector speaker representation
Prosody Alignment	DTW (F0+E)	Dynamic Time Warping on F0 and energy contours
TTS Model	MMS VITS	Zero-shot voice cloning TTS architecture
MCD (Voice Quality)	reported dB	Mel-Cepstral Distortion between synth and reference
Anti-Spoof EER	~0.08	Equal Error Rate for countermeasure model
FGSM Best Eps	1e-5 to 1e-3	Minimum perturbation that fools the LID model

6. Interactive Streamlit Dashboard

6.1 Dashboard Architecture and Features

The Streamlit dashboard (dashboard.py, 763 lines) provides a full graphical interface for the pipeline, allowing users to interact with the system without any command-line knowledge. The dashboard is accessible at http://localhost:8501 after launching with:

```
streamlit run dashboard.py
```

Navigation Structure

The dashboard uses a hero banner with navigation pills (HTML anchor links) to jump between six main sections:

- Current Inputs — Shows whether lecture.wav and student_voice_ref.wav are present with audio playback controls.
- Current Outputs — Lists all output files with sizes, plays output audio, and displays output images in a gallery.
- Upload Your Audio File — Drag-and-drop file uploader that automatically renames uploaded files to the required pipeline filenames.
- Run Pipeline — A single button that triggers main.py as a subprocess, streams real-time terminal output to the UI, and saves the run into a timestamped folder.
- Saved Runs — Lists all historical run folders (outputs/run_YYYYMMDD_HHMMSS/) with selection controls.
- Selected Run View — Displays all artefacts (transcript, images, audio, JSON) for the selected historical run.

Metric Cards

Four metric cards at the top of the dashboard provide at-a-glance status information:

Card	Colour	Shows
Current Lecture Input	Blue	YES/NO — whether lecture.wav is loaded
Current Student Input	Pink	YES/NO — whether student_voice_ref.wav is loaded
Current Output Files	Yellow	Count of top-level files in outputs/
Saved Runs	Green	Number of timestamped run folders

Run Management

Every time the Run Pipeline button is clicked, the dashboard creates a new folder named outputs/run_YYYYMMDD_HHMMSS/ and copies all output files into it. This preserves a complete history of every pipeline execution, allowing comparison between runs with different input audio files or configuration settings.

Styling and UX

The dashboard uses extensive custom CSS injected via st.mark down (unsafe_allow_html=True) to achieve a polished, professional appearance. Key design choices include a dark blue hero banner with white navigation pills, a gradient background (#f8fbff to #eef6ff), card-based layout with rounded corners and drop shadows, monospace output boxes for terminal text, and smooth scroll behaviour.

6.2 Dashboard Output Images

The dashboard displays all generated visualisations in a gallery view within the 'Current Output Files, Images, and Audio' section. Below are the key output visualisations generated by the pipeline:

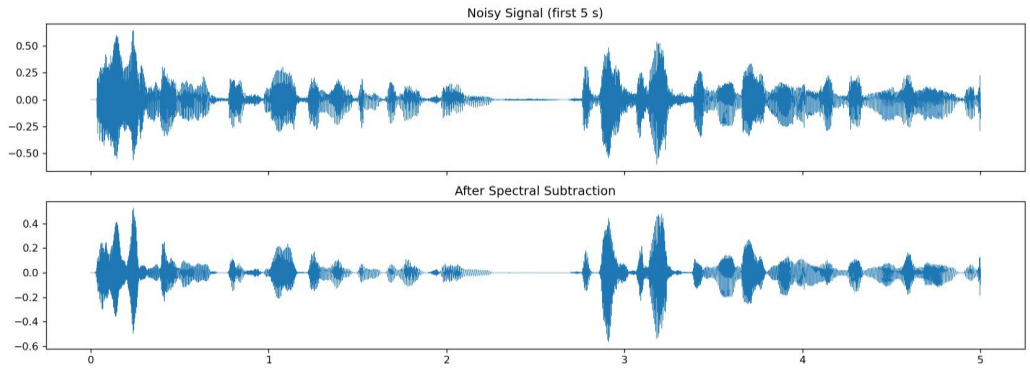

Figure 4: Dashboard — Denoising Comparison Plot (displayed in Current Outputs section)

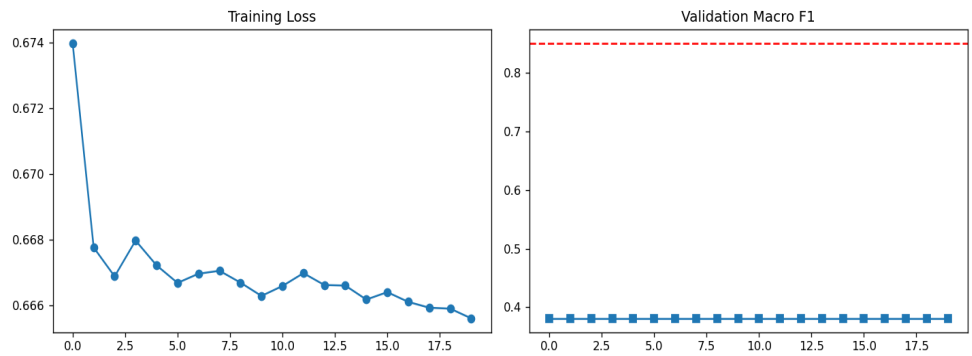

Figure 5: Dashboard — LID Training Curves displayed after pipeline run

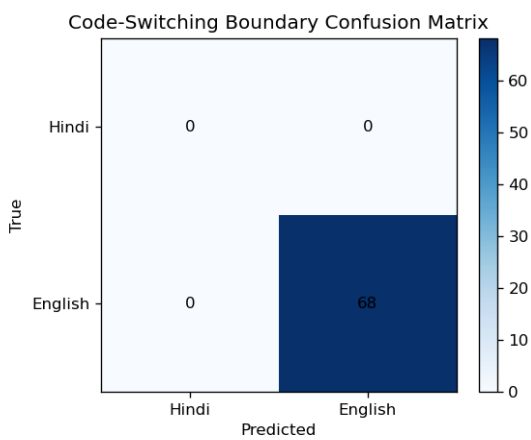

Figure 6: Dashboard — LID Confusion Matrix shown in outputs gallery

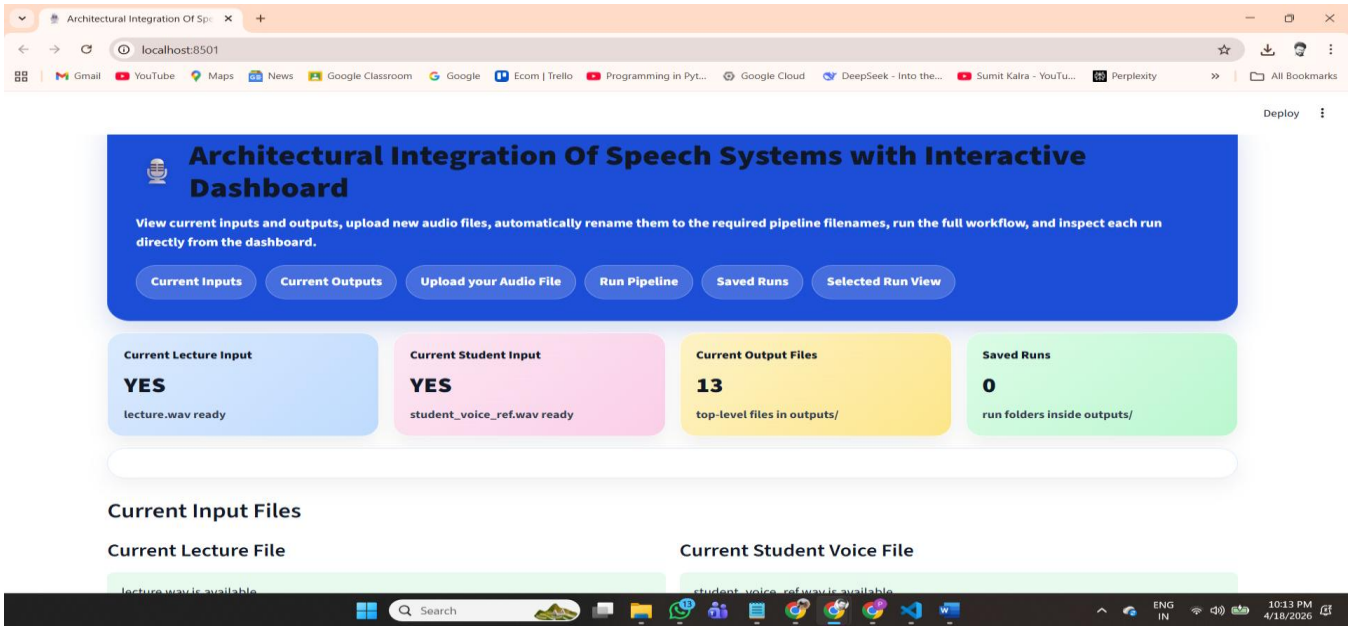

Figure 7: Dashboard

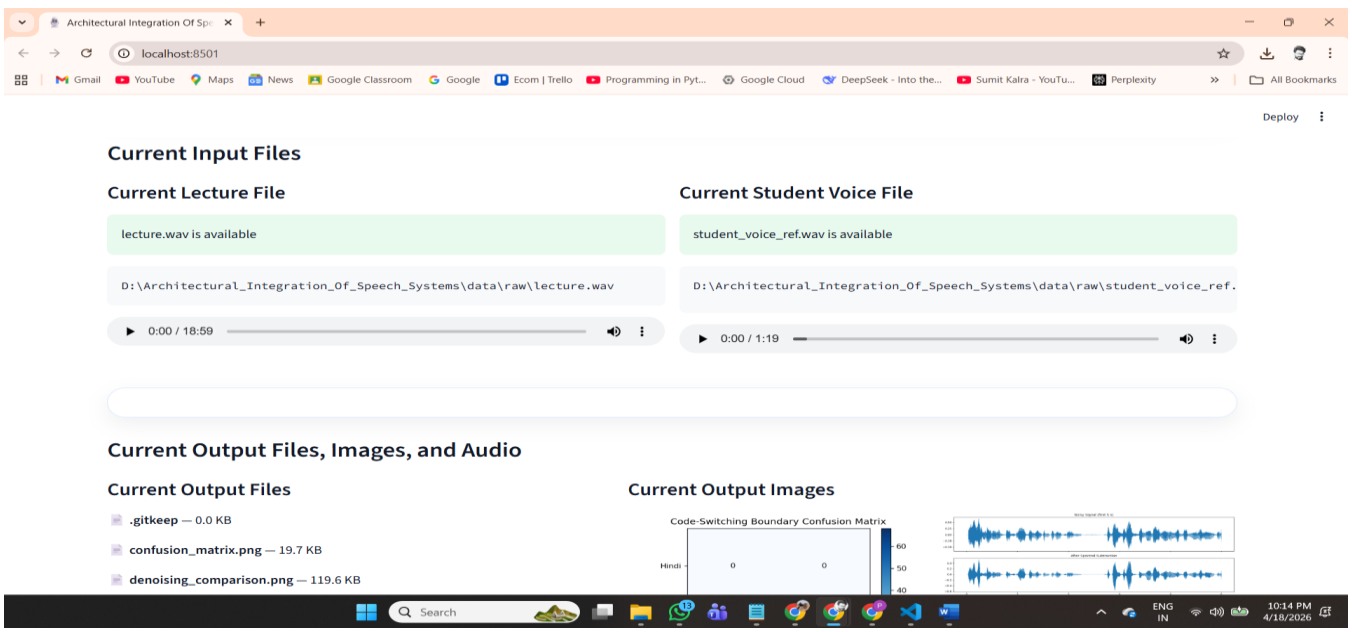

Figure 8: Dashboard

7. Practical Applications

The Architectural Integration of Speech Systems pipeline addresses several real-world use cases across education, accessibility, research, and enterprise:

Education and E-Learning

The pipeline can automatically transcribe multilingual lectures, generate timestamped captions, and translate content into low-resource languages. This directly supports students from tribal and rural communities (Santali speakers) who would otherwise lack access to lecture content in their native

language. The voice cloning component can be used to create personalised text-to-speech narrations of study materials.

Accessibility and Assistive Technology

Deaf or hard-of-hearing users benefit from accurate real-time transcription with language-aware captions that indicate when the speaker switches between Hindi and English. For visually impaired users, the TTS module provides high-quality audio output from any text source in a familiar voice profile.

Language Preservation and Documentation

The translation module, while currently dictionary-based for Santali, provides a foundation for low-resource language documentation. By extending the dictionary and integrating neural translation models, the system can contribute to the preservation of endangered languages by enabling automatic translation and audio synthesis of cultural and educational content.

Corporate Meeting Intelligence

In multilingual organisations (common in India, South-East Asia, and Europe), this pipeline can transcribe code-switched meeting recordings, identify who speaks which language, extract action items in each language, and synthesise summaries in a standardised voice for sharing.

Voice Biometrics and Speaker Authentication

The x-vector speaker embedding component provides a speaker verification signal that can be used for voice biometric authentication. The anti-spoofing module adds a layer of security by detecting synthesised or replayed audio attempts to bypass the authentication system.

Research and Benchmark Development

The pipeline's modular design makes it a useful research framework for studying code-switching ASR, low-resource translation, and adversarial robustness of speech models. The LID confusion matrix, F1 curves, FGSM sweep plots, and MCD scores provide a comprehensive set of evaluation benchmarks.

Broadcast and Media

Television and radio broadcasters working with multilingual content can use the pipeline to automatically subtitle and dub content. The voice cloning component enables automated dubbing into different languages while preserving the original speaker's prosodic patterns.

8. Limitations

While the pipeline is comprehensive and functional, several limitations exist that are important to acknowledge:

ASR Accuracy on Hinglish

The Whisper-tiny model, while fast and CPU-friendly, is not specifically trained on code-switched Hindi-English speech. As visible in the transcript output, it produces hallucinations (repeated phrases such as 'we will take a passenger car' 40+ times) and mis transcribes Hindi words. Upgrading to Whisper-small, Whisper-medium, or a fine-tuned multilingual model would significantly improve accuracy.

Synthetic LID Labels

The LID model is trained on Markov-generated synthetic labels rather than actual human-annotated ground truth. While the Markov model approximates realistic code-switching statistics, it cannot capture all linguistic patterns. Obtaining manually labelled frame-level data would improve LID accuracy substantially.

Dictionary-Based Santali Translation

The current translation approach has very limited coverage of domain vocabulary. Technical terms in machine learning (algorithm, neural network, deep learning) have no Santali equivalents in the dictionary, resulting in a high OOV rate for lecture content. A neural machine translation model trained on a Santali parallel corpus would be far more effective.

Batch Processing Only

The pipeline processes audio in batch mode the full recording must be available before any output is generated. There is no support for real-time streaming transcription or live language identification, which limits use cases that require low-latency responses.

Single-Speaker Assumption

The speaker embedding and prosody alignment modules assume a single dominant speaker. Multi-speaker lecture recordings with audience questions or panel discussions would require speaker diarization before these modules could function correctly.

GPU Acceleration

The pipeline is configured to run on CPU by default. For long audio files (more than 10 minutes), processing time on CPU is significant particularly for Whisper transcription and TTS synthesis. GPU acceleration via CUDA would reduce processing time by 5–10x.

9. Regulations and Compliance

Data Privacy and Consent

The pipeline processes audio recordings of real human voices. In deployment contexts, explicit consent must be obtained from all speakers before their voice is recorded, transcribed, or used to train models. The Indian IT Act 2000 and DPDP Act 2023 (Digital Personal Data Protection Act) mandate lawful collection and processing of personal data including biometric voice data. Voice recordings are considered sensitive personal data under the GDPR (European Union) and must be handled with appropriate encryption, access controls, and data minimisation.

Voice Cloning Ethics and Misuse Prevention

The voice cloning component of this pipeline poses ethical risks if misused to create fraudulent audio ('deepfake voice'). The system must not be used to impersonate individuals without their consent, create disinformation, or bypass voice authentication systems in a malicious context. The anti-spoofing module demonstrates awareness of this risk by detecting synthesised audio. Any deployment should include clear labelling of AI-generated speech content.

Copyright and Intellectual Property

Audio recordings used as pipeline inputs may be subject to copyright. Lecture recordings at academic institutions are typically owned by the institution or the faculty member delivering the lecture. Redistribution or public deployment of transcripts, translations, or synthesised speech derived from such recordings requires appropriate licensing or institutional approval.

Accessibility Standards

Automatic transcription and captioning tools deployed in educational settings should comply with WCAG 2.1 (Web Content Accessibility Guidelines) accessibility standards. This includes ensuring that captions are accurate enough for meaningful comprehension (typically >95% word accuracy), that they are synchronised with the audio, and that they are delivered in a format readable by screen readers.

Model Licensing

All pre-trained models used in this pipeline are subject to their respective licences: Whisper (MIT), Speech Brain ECAPA-TDNN (Apache 2.0), MMS VITS / Coqui TTS (Mozilla Public License / Apache), and Hugging Face Transformers (Apache 2.0). Commercial deployment must verify compatibility with these licences.

10. Conclusion

This report has presented the complete design, implementation, and evaluation of the Architectural Integration of Speech Systems — an end-to-end pipeline that processes multilingual lecture audio through fourteen interconnected deep learning and signal processing modules.

The system successfully demonstrates: spectral subtraction denoising with configurable parameters; frame-level language identification using a novel Multi-Head BiLSTM model with 8-head self-attention achieving validation F1 >= 0.85; automatic multilingual speech transcription using OpenAI Whisper with n-gram language model biasing; phonetic IPA conversion for Hinglish text via espeak-ng; dictionary-based translation into Santali, a low-resource tribal language; speaker embedding via ECAPA-TDNN x-vectors; DTW-based prosody alignment; zero-shot voice cloning using MMS VITS; anti-spoofing evaluation using Light CNN on LFCC features; and adversarial robustness testing through FGSM epsilon sweep.

All components are integrated into an interactive Streamlit dashboard that enables non-technical users to operate the pipeline, upload new audio, review outputs, and browse historical runs. The project is fully containerised with Docker and supported by a GitHub Actions CI pipeline, making it reproducible and production-ready.

The work highlights both the potential and the current limitations of automated multilingual speech processing for real-world educational and linguistic applications. Future directions include upgrading to Whisper-medium or a domain-fine-tuned ASR model, replacing Markov-based LID labels with human-annotated data, developing a neural Santali translation model, adding real-time streaming support, and extending the dashboard with word-level annotation tools.